Tydeus-16

Core Description Document

v0.1

Contents

1 Goal **3**

2 Specifications **4**

3 Registers **5**
3.1 Zero register . . . 5
3.2 General-purpose registers . . . 5
3.3 Program counter register . . . 5
3.4 Stack pointer register . . . 5
3.5 Flags register . . . 5

4 Instruction Set Architecture (ISA) **6**
4.1 Instruction formats . . . 6
4.2 No-operation instructions . . . 7
4.2.1 NOP . . . 7
4.3 Data movement instructions . . . 7
4.3.1 MOV . . . 7
4.3.2 LI . . . 7
4.3.3 LIH . . . 8
4.4 Memory instructions . . . 8
4.4.1 LOAD . . . 8
4.4.2 STORE . . . 8
4.5 Arithmetic and logic instructions . . . 8
4.5.1 ADD . . . 8
4.5.2 ADDI . . . 8
4.5.3 SUB . . . 9
4.5.4 SUBI . . . 9
4.5.5 AND . . . 9
4.5.6 OR . . . 9
4.5.7 XOR . . . 9
4.5.8 NOT . . . 10
4.5.9 CMP . . . 10
4.5.10 CMPI . . . 10
4.6 Shift instructions . . . 10
4.6.1 SLL . . . 10
4.6.2 SRL . . . 10
4.6.3 SRA . . . 11
4.7 Jump instructions . . . 11
4.7.1 JMP . . . 11
4.7.2 BEQ . . . 11
4.7.3 BNE . . . 11
4.7.4 BLT . . . 11
4.7.5 BGT . . . 11
4.7.6 BLE . . . 12
4.7.7 BGE . . . 12
4.8 Flow control instructions . . . 12
4.8.1 CALL . . . 12
4.8.2 RET . . . 12
4.9 Instruction table . . . 13

5 Memory **14**
5.1 Instruction memory 14
5.2 Data memory 14
5.3 Stack memory organization 14
5.4 Pending memory-related decisions 15

6 Datapath **16**

1. Goal

2. Specifications

- Data width: 16 bits
- Instruction width: 16 bits
- General-purpose registers: 7
- Single clock
- No interrupts
- Multi-cycle control unit
- Separated instruction and data memories

3. Registers

The processor exposes a compact architectural register set composed of one zero register, seven general-purpose registers, the program counter, the stack pointer, and a flags register.

3.1 Zero register

`R0` is a 16-bit constant zero register. Its value is permanently fixed to `0x0000`, and any write to `R0` is ignored.

3.2 General-purpose registers

The processor provides seven 16-bit general-purpose registers, R1 to R7. These registers are used as operands and destinations for the instructions.

3.3 Program counter register

The program counter, PC, is an 11-bit register that holds the address of the current instruction in instruction memory.

> **Note** PC register is 11-bit wide due to the instruction memory size being 2048 words. See more information on section 5.1.

3.4 Stack pointer register

The stack pointer, SP, is a 16-bit register that points to the element currently located at the top of the stack.

3.5 Flags register

The FLAGS register is a 4-bit register that stores the processor's ALU status flags. It contains the following four 1-bit elements:

- Z: zero flag
- C: carry flag
- N: negative flag
- V: overflow flag

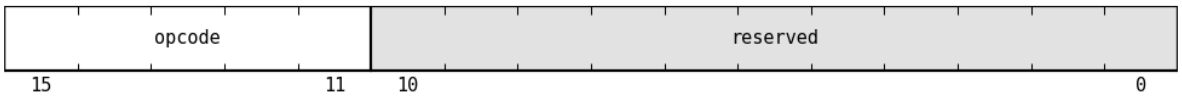
4. Instruction Set Architecture (ISA)

This chapter defines the processor instruction set architecture, including the available instruction formats, the supported instructions, and the `opcode` allocation used by the instruction decoder.

The ISA uses fixed-width 16-bit instructions. Depending on the operation, instructions may encode register operands, immediates, absolute addresses, or relative branch offsets. The instruction set is intentionally compact and is organized into a small number of encoding formats.

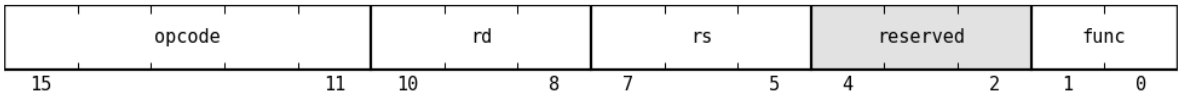
4.1 Instruction formats

No operand: `NOP`, `RET`

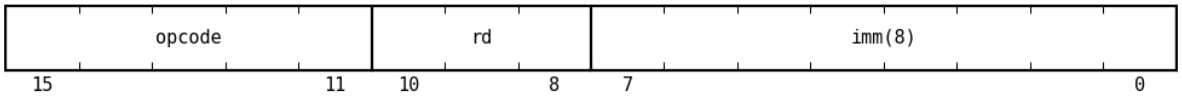


Register-register-register (RRR): `ADD`, `SUB`, `AND`, `OR`, `XOR`

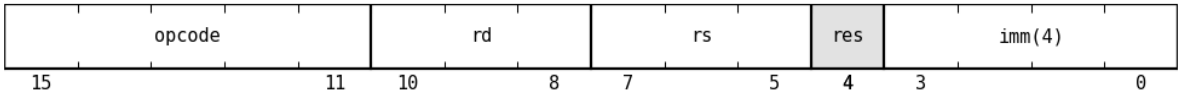
Register-register (RR): `MOV`, `NOT`, `CMP`



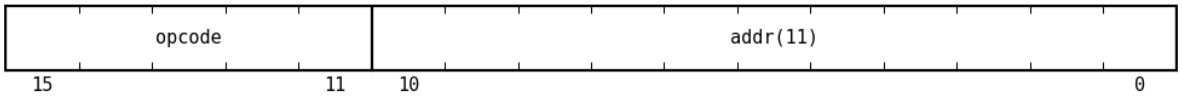
Register-immediate (RI): `LI`, `LIH`, `ADDI`, `SUBI`, `CMPI`



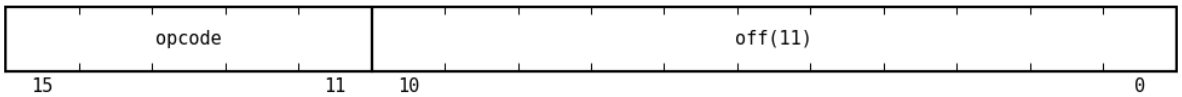
Register-register-immediate (RRI): `SLL`, `SRL`, `SRA`



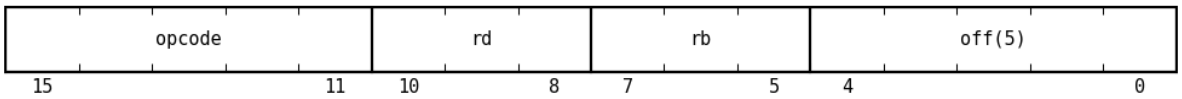
Unconditional jump: `JMP`, `CALL`



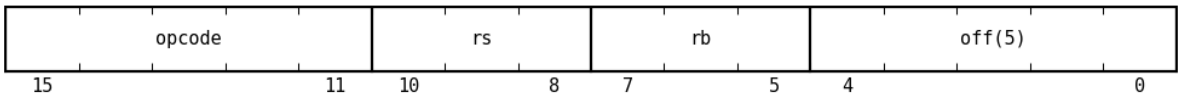
Conditional branch: `BEQ, BNE, BLT, BGT, BLE, BGE`



Load: `LOAD`



Store: `STORE`



4.2 No-operation instructions

4.2.1 NOP

The `NOP` instruction performs no operation and does not modify the architectural state of the processor.

```
NOP                 // does nothing
```

4.3 Data movement instructions

4.3.1 MOV

The `MOV` instruction copies the contents of a source register into a destination register.

```
MOV rd, rs          // rd <- rs
```

4.3.2 LI

The `LI` instruction loads an 8-bit immediate into the lower half of the destination register and clears the upper half to zero. This instruction is used to efficiently load small constants.

```
LI rd, imm(8)       // rd[7:0] <- imm(8), rd[15:8] <- 0x00
```

4.3.3 LIH

The `LIH` instruction loads an 8-bit immediate into the upper half of the destination register while leaving the lower half unchanged. Combined with `LI`, it allows constructing a full 16-bit constant.

> **Note** To successfully load a 16-bit immediate into a register it is mandatory to use `LI` first and then `LIH`.

```
LIH rd, imm(8)      // rd[15:8] <- imm(8)
```

4.4 Memory instructions

4.4.1 LOAD

The `LOAD` instruction reads a word from data memory using BASE + `offset` addressing. The effective address is computed as the contents of the base register plus the sign-extended 5-bit offset.

```
LOAD rd, off(5)[rb]     // rd <- MEM[rb + sext(off(5))]
```

4.4.2 STORE

The `STORE` instruction writes a word into data memory using BASE + `offset` addressing. The effective address is computed as the contents of the base register plus the sign-extended 5-bit offset.

```
STORE off(5)[rb], rs    // MEM[rb + sext(off(5))] <- rs
```

4.5 Arithmetic and logic instructions

4.5.1 ADD

The `ADD` instruction adds two source registers and writes the result to the destination register.

```
ADD rd, rs1, rs2        // rd <- rs1 + rs2
```

4.5.2 ADDI

The `ADDI` instruction adds a sign-extended 8-bit immediate to the destination register.

```
ADDI rd, imm(8)         // rd <- rd + sext(imm(8))
```

4.5.3 SUB

The SUB instruction subtracts the second source register from the first source register and writes the result to the destination register.

```
SUB rd, rs1, rs2    // rd <- rs1 - rs2
```

4.5.4 SUBI

The SUBI instruction subtracts a sign-extended 8-bit immediate from the destination register.

```
SUBI rd, imm(8)     // rd <- rd - sext(imm(8))
```

4.5.5 AND

The AND instruction performs a bitwise AND between two source registers.

```
AND rd, rs1, rs2    // rd <- rs1 and rs2
```

> **Note** Logical operations (AND, OR, XOR, NOT) do not have immediate versions because an 8-bit immediate is not especially useful in a 16-bit architecture, particularly for full-width bit masks. These instructions may be added in the future if free opcodes remain available.

4.5.6 OR

The OR instruction performs a bitwise OR between two source registers.

```
OR rd, rs1, rs2     // rd <- rs1 or rs2
```

4.5.7 XOR

The XOR instruction performs a bitwise exclusive OR between two source registers.

```
XOR rd, rs1, rs2    // rd <- rs1 xor rs2
```

4.5.8 NOT

The NOT instruction performs a bitwise negation of the source register.

```
NOT rd, rs          // rd <- not rs
```

4.5.9 CMP

The CMP instruction behaves like a subtraction operation, but the result is not written back to the register file. Instead, only the status flags are updated.

```
CMP  rd, rs         // rd - rs
```

4.5.10 CMPI

The CMPI compares the register operand with a sign-extended 8-bit immediate and acts as the CMP instruction.

```
CMPI rd, imm(8)     // rd - sext(imm(8))
```

4.6 Shift instructions

4.6.1 SLL

The SLL instruction performs a logical left shift of the source register by an immediate amount.

```
SLL rd, rs, imm(4)  // rd <- rs << imm(4)
```

> **Note** Shift instructions use only a 4-bit immediate because this is sufficient to encode all meaningful shift amounts in a 16-bit architecture. A 4-bit field allows values from 0 to 15, which covers the full useful shift range for a 16-bit operand.

4.6.2 SRL

The SRL instruction performs a logical right shift of the source register by an immediate amount.

```
SRL rd, rs, imm(4)  // rd <- rs >> imm(4)
```

4.6.3 SRA

The SRA instruction performs an arithmetic right shift of the source register by an immediate amount.

```
SRA rd, rs, imm(4)  // rd <- rs >> imm(4) ; arithmetic
```

4.7 Jump instructions

4.7.1 JMP

The JMP instruction performs an unconditional jump to an absolute 11-bit address in instruction memory.

```
JMP addr(11)        // PC <- addr(11)
```

4.7.2 BEQ

The BEQ instruction performs a relative branch if the zero flag is set.

```
BEQ off(11)         // if Z=1 then PC <- PC+1 + off(11)
```

4.7.3 BNE

The BNE instruction performs a relative branch if the zero flag is not set.

```
BNE off(11)         // if Z=0 then PC <- PC+1 + off(11)
```

4.7.4 BLT

The BLT instruction performs a relative branch if the signed comparison indicates strictly less than.

```
BLT off(11)         // if (N xor V)=1 then PC <- PC+1 + off(11)
```

4.7.5 BGT

The BGT instruction performs a relative branch if the signed comparison indicates strictly greater than.

```
BGT off(11)          // if Z=0 and (N xor V)=0 then PC <- PC+1 + off(11)
```

4.7.6 BLE

The BLE instruction performs a relative branch if the signed comparison indicates less than or equal.

```
BLE off(11)          // if Z=1 or (N xor V)=1 then PC <- PC+1 + off(11)
```

4.7.7 BGE

The BGE instruction performs a relative branch if the signed comparison indicates greater than or equal.

```
BGE off(11)          // if (N xor V)=0 then PC <- PC+1 + off(11)
```

4.8 Flow control instructions

4.8.1 CALL

The CALL instruction saves the return address on the stack and then transfers control to an absolute 11-bit address in instruction memory.

```
CALL addr(11)        // SP <- SP-1 ; MEM[SP] <- PC+1 ; PC <- addr(11)
```

4.8.2 RET

The RET instruction restores the program counter from the stack and returns execution to the caller.

```
RET                  // PC <- MEM[SP] ; SP <- SP+1
```

4.9 Instruction table

Instruction	opcode	func	Format
NOP	00000	-	No operand
MOV	00001	00	Register-register (RR)
ADD	00001	01	Register-register-register (RRR)
SUB	00001	10	Register-register-register (RRR)
CMP	00001	11	Register-register (RR)
AND	00010	00	Register-register-register (RRR)
OR	00010	01	Register-register-register (RRR)
XOR	00010	10	Register-register-register (RRR)
NOT	00010	11	Register-register (RR)
LI	00011	-	Register-immediate (RI)
LIH	00100	-	Register-immediate (RI)
SLL	00101	-	Register-register-immediate (RRI)
SRL	00110	-	Register-register-immediate (RRI)
SRA	00111	-	Register-register-immediate (RRI)
JMP	01000	-	Unconditional jump
BEQ	01001	-	Conditional branch
BNE	01010	-	Conditional branch
BLT	01011	-	Conditional branch
BGT	01100	-	Conditional branch
BLE	01101	-	Conditional branch
BGE	01110	-	Conditional branch
CALL	01111	-	Unconditional jump
RET	10000	-	No operand
Reserved	10001	-	-
LOAD	10010	-	Load
STORE	10011	-	Store
ADDI	10100	-	Register-immediate (RI)
SUBI	10101	-	Register-immediate (RI)
CMPI	10110	-	Register-immediate (RI)
Reserved	10111	-	-
Reserved	11xxx	-	-

Table 4.1: Instruction encoding table

5. Memory

The processor uses a Harvard-style memory organization, with separated instruction and data memories. Both memories are word-addressed, use 16-bit words, and are aligned to word boundaries. The system is little-endian and both memories are assumed to have a latency of one cycle. These decisions simplify the implementation of the control unit and avoid structural hazards between instruction fetches and data accesses.

5.1 Instruction memory

The instruction memory stores the program code executed by the processor. It contains $2^{11} = 2048$ words of 16 bits each, which corresponds to a total size of 4096 bytes (4 KB).

The instruction memory size directly constrains the width of the PC register to 11 bits, since the processor can address up to 2048 instruction words. Unconditional jump instructions such as JMP and CALL also use absolute 11-bit addresses, while conditional branches use a signed 11-bit offset relative to PC+1.

> **Note** Instruction memory is read-only from the point of view of the ISA. No instruction in the current instruction set modifies instruction memory.

5.2 Data memory

The data memory stores program data, stack contents, and any other memory-resident values manipulated by the processor. It contains $2^{16} = 65536$ words of 16 bits each, which corresponds to a total size of 131072 bytes (128 KB). Therefore, the data memory address space is 16 bits wide.

Memory accesses are performed at word granularity. In the current version of the ISA, data memory is accessed only through BASE + `offset` addressing:

```
LOAD  rd, off(5)[rb]
STORE off(5)[rb], rs
```

In both instructions, the effective address is computed as the contents of the base register plus the sign-extended 5-bit offset. Offsets are signed, which allows both forward and backward accesses relative to the base register.

5.3 Stack memory organization

The stack resides in the upper region of data memory and grows towards lower addresses. The SP register always points to the element currently located at the top of the stack. After reset, SP is initialized to the end of data memory. Function calls and returns are implemented using this stack mechanism:

- `CALL` decrements SP and stores the return address in `MEM[SP]`.
- `RET` loads the return address from `MEM[SP]` and then increments SP.

This stack model is sufficient to support subroutine calls in the current version of the processor.

5.4 Pending memory-related decisions

Some memory behaviors are intentionally left open for future refinement. In particular, the following points may be defined later in the project:

- the behavior on accesses outside the valid data memory range,
- the behavior on accesses to regions reserved for stack usage,
- and whether any memory-mapped I/O region will be introduced.

At the current stage, the ISA and memory system are defined only to the extent required to support program execution, stack operation, and basic data access.

6. Datapath